Module 01 — Python Setup and First Programs

Phase: Core Python
Previous: Computer and Programming Foundations
Next: Variables, Datatypes, Operators, and I/O

Module Outcome

Install Python 3, run code in the REPL and in a `.py` file, and ship a short welcome script with `print`, comments, clear names, and a simple f-string.

By the end of this module you will be able to:

- Verify Python 3 from the terminal and pick that interpreter in VS Code
- Choose REPL for a one-line check and a script for work you keep
- Print text, numbers, and a banner with `"=" * n`
- Name values with `snake_case` and avoid keywords
- Write a one-line f-string greeting

1. Why This Concept Matters

Every later lab is a **file you run**. If Python is missing, or you only type in an unsaved tab, nothing in this course will work on your laptop.

Python is a **high-level** language: you write readable lines; the **interpreter** runs them. One language covers small scripts now and APIs / Django later, so class time goes to **logic**, not boilerplate.

A campus portal splash screen is the same idea as `Hello, World!`: text on the screen, no database yet.

```
print("Hello, World!")
```

Expected Output:

```
Hello, World!
```

That one line is a complete program.

2. Install Python 3 and verify

Understand

Your machine needs a **Python 3** interpreter and an editor that can run it. Without that pair, later modules cannot run on your laptop.

How It Works

1. Download **Python 3.12+** from python.org/downloads.
   Windows: tick **Add python.exe to PATH**.

2. In VS Code, install the **Python** extension.
3. Command Palette → **Python: Select Interpreter** → pick Python 3.x.
4. Open your course folder. Create a `projects/` folder for scripts.

```
# macOS / Linux
python3 --version

# Windows (often)
python --version
```

You should see `Python 3.12.x` (or another 3.x). Save that line — the mini-project asks for it.

A tiny script proves the editor is using the same interpreter:

```
import sys

print("Python version:", sys.version)
print("Setup OK — ready for Core Python!")
```

Expected Output: (the version text will match *your* machine)

```
Python version: 3.12.x ...
Setup OK — ready for Core Python!
```

⚠️ WATCH OUT

```
# Wrong — VS Code running a different Python than the one you installed
# Prefer — status bar → select the Python 3.x you just verified in the terminal
```

Python 2 `print "hi"` is obsolete. This course uses `print("hi")` on Python 3.

Try It

In the terminal, print your Python version. Then create `projects/env_check.py` with the two `print` lines above and run it.

---

3. REPL vs script

Understand

Two places to run Python:

- **REPL** — interactive prompt (`>>>`). Type a line, see the result. Good for a quick check.
- **Script** — a saved `.py` file. You run it with `python3 file.py`. Good for assignments you keep and submit.

Habit: try a line in the REPL → copy working lines into a `.py` file → run the script.

How It Works

Mode	How	When
REPL	Type `python3` → see `>>>`	One-line checks, calculator

Mode	How	When
Script	Save `.py`, run `python3 hello.py`	Assignments, projects

```
python3
```

```
>>> print("Hello from REPL")
Hello from REPL
>>> 2 + 3
5
>>> exit()
```

Script (`projects/hello.py`):

```
# hello.py
# Purpose: First Python program

print("Hello, World!")
print("Welcome to Python Full Stack training.")
```

```
python3 hello.py
```

Expected Output:

```
Hello, World!
Welcome to Python Full Stack training.
```

`print` can show a calculated number too — the REPL-style check, saved in a file:

```
print("Hello, Ravi")
print(450 + 120)
```

Expected Output:

```
Hello, Ravi
570
```

⚠ WATCH OUT

```
# Wrong — run without saving (tab still has a dot) → old output
# Prefer — Save (Cmd+S / Ctrl+S) before you run

# Wrong — terminal is in the wrong folder → FileNotFoundError
# Prefer — cd into the folder that contains the script

# Wrong
print(Hello)          # NameError — Hello looks like a variable
```

```
# Prefer
print("Hello")
```

Try It

Create `projects/hello.py`. Print a greeting for your name and the sum of `199 + 49`.

---

4. Comments and banners

Understand

`#` starts a **comment** — Python ignores it. Comments explain *why* for humans.

Repeating a character with `"=" * 40` draws a border. Several `print` lines stacked together look like a portal or ATM screen.

How It Works

```
# Purpose: Career intro line
print("Priya")
print("Become a full-stack Python developer")
```

Expected Output:

```
Priya
Become a full-stack Python developer
```

```
print("=" * 40)
print("   KSR STUDENT PORTAL")
print("=" * 40)
print("Login page loading...")
print("Please use registered email.")
print("=" * 40)
```

Expected Output:

```
========================================
   KSR STUDENT PORTAL
========================================
Login page loading...
Please use registered email.
========================================
```

⚠ WATCH OUT

Do not put passwords or secrets in comments you will commit. Keep comments short: `# Purpose: …`

`"=" * 40` is 40 equal signs — change the number if the title is shorter or longer.

Try It

Print a 4-line canteen banner: `=` border, title `CAMPUS CANTEEN`, a welcome line, `=` border.

---

5. Keywords and identifiers

Understand

Keywords are reserved words (`if`, `class`, `for`, `True`, …). You cannot use them as names.

Identifiers are names you choose. This week they label values (later they will also name functions).

Rules:

- Start with a letter or `_`
- Then letters, digits, `_`
- Case-sensitive: `name` is not `Name`
- No spaces → `student_name`
- Style: **snake_case** for variables

How It Works

```
# INVALID — SyntaxError if you uncomment these
# if = 10
# class = "A"

# VALID
student_count = 45
employee_id = "EMP001"
full_name = "Ananya Sharma"
is_active = True

print(full_name)
print(employee_id)
print(is_active)
```

Expected Output:

```
Ananya Sharma
EMP001
True
```

⚠ WATCH OUT

```
# Wrong
class = "Python"       # keyword
employee id = 101      # space
FullName = "Ananya"    # works, but not snake_case for a variable

# Prefer
course_name = "Python"
employee_id = 101
full_name = "Ananya"
```

Try It

Store a student name, a roll number, and a pass flag (`True` or `False`) with `snake_case` names. Print each.

6. Formatted greeting (f-string preview)

Understand

An **f-string** embeds values inside `{ }` in a string that starts with `f`. Next module covers formatting in more depth — today you only need “put this name in this sentence.”

How It Works

```
trainer_name = "Kasi"
batch_name = "Python Full Stack"

print("Hello,", trainer_name)
print(f"Hello, {trainer_name}! Batch: {batch_name}.")
```

Expected Output:

```
Hello, Kasi
Hello, Kasi! Batch: Python Full Stack.
```

`print("Hello,", trainer_name)` joins pieces with a space. The f-string keeps the whole sentence in one string.

⚠️ WATCH OUT

```
# Wrong — missing f  →  the braces print as text
print("Hello, {trainer_name}")

# Prefer
print(f"Hello, {trainer_name}")
```

Try It

Store a name and a city. Print `{name} from {city}` with an f-string.

Step-by-Step: Build It

One day-1 intro card: purpose comment, `snake_case` values, banner, and an f-string. Type the file. Do not paste.

```
# Purpose: Day 1 intro card

student_name = "Priya Sharma"
city = "Hyderabad"
goal = "Become a full-stack Python developer"

print("=" * 36)
print("  KSR — Day 1 Intro")
print("=" * 36)
print(f"Name: {student_name}")
print(f"City: {city}")
print(f"Goal: {goal}")
print("=" * 36)
```

Expected Output:

```
====================================
  KSR — Day 1 Intro
====================================
Name: Priya Sharma
City: Hyderabad
Goal: Become a full-stack Python developer
====================================
```

Save as `projects/my_intro.py` and run `python3 my_intro.py`. Swap in your own name, city, and goal.

Tonight's kit also asks you to prove the install and keep a hello script — same tools, a small **folder pack**, not this one file.

Practice Exercises

Attempt each problem before reading the model answer.

1. Print a 3-line train ticket stub: passenger `Meera Rao`, route `Pune → Hyderabad`, date `14-09-2026`.

Hint: Three `print` calls. The date can be a string.

Model answer:

```
print("Meera Rao")
print("Pune → Hyderabad")
print("14-09-2026")
```

Expected Output:

```
Meera Rao
Pune → Hyderabad
14-09-2026
```

2. Store `book = "Atomic Habits"` and `desk = 12`. Print `Hold: Atomic Habits — desk 12` with an f-string.

Model answer:

```
book = "Atomic Habits"
desk = 12
print(f"Hold: {book} — desk {desk}")
```

Expected Output:

```
Hold: Atomic Habits — desk 12
```

3. Print a 5-line library banner: `"=" * 28` borders, title `KSR LIBRARY`, then `Quiet zone`.

Model answer:

```
print("=" * 28)
print("      KSR LIBRARY")
```

```
print("=" * 28)
print("  Quiet zone")
print("=" * 28)
```

Expected Output:

```
============================
      KSR LIBRARY
============================
  Quiet zone
============================
```

4. Invent valid `snake_case` names for the invalid ideas `for`, `roll no`, and `BatchName`. Assign a course, a roll, and a batch, then print all three.

Hint: `for` is a keyword. Spaces are not allowed. Variables use `snake_case`, not `PascalCase`.

Model answer:

```
loop_count = 3
roll_no = "2026001"
batch_name = "PFS-A"
print(loop_count)
print(roll_no)
print(batch_name)
```

Expected Output:

```
3
2026001
PFS-A
```

5. Using only `print` and arithmetic (no variables required), print remaining seats: `120 - 47`.

Model answer:

```
print(120 - 47)
```

Expected Output:

```
73
```

Lab

Combine comments, `snake_case` names, a banner, an f-string, and a printed total. One file — not the folder pack from the kit.

Problem

A campus desk needs a **welcome card** on screen: who the student is, which batch, and what they owe for the week.

What to Build

A script `welcome_card.py`.

Requirements

- A `# Purpose:` comment at the top
- Variables: `student_name`, `batch_name`, `course_fee`, `lab_fee` (`snake_case`)
- A banner with `"=" * n` around a short title
- An f-string that greets the student and shows the batch
- Print the **sum** of the two fees

Expected Behavior

Running the script prints a bordered card. The fee line is a calculated total, not a typed-in number you already added by hand in your head only — let Python add them.

Sample Run

```
========================
  CAMPUS WELCOME
========================
Hello, Dev Kumar — batch PFS-A
Fees due: 4500
========================
```

(Your name, batch, and amounts may differ; the **shape** should match.)

Constraints

Top-level statements only — no `if`, no loops, no `def`. This is one `.py` file, not a folder layout.

Bonus

Add a city variable and a second f-string line: `{student_name} from {city}`.

Mini-Project (Assignment): Environment and Greeting Pack

Why this matters: Mentors need proof that your machine runs Python 3 and that you can save and run scripts — the same habit every later lab uses.

Lab kit: 15_Lab_Assets/27_terminal_workspace/

File / area	What you do
Kit checklist	Follow `starter/CHECKLIST.md` — create the standard folders
`notes/day1.txt`	Paste `pwd` + listing, plus `python3 --version` (or `python --version`)
`projects/env_check.py`	Print Python version + ready message
`projects/hello.py`	Hello World + welcome line
`projects/my_intro.py`	Your name, career goal, why Python full stack

File / area	What you do
`solution/`	Do **not** copy this

This is a **folder pack** — not the single welcome-card lab. No `if` / loops yet.

Objective

Create the course workspace folders and three small scripts that prove install + `print`.

Story / Scenario

Day 1 of training. A classmate should see that your laptop runs Python 3 and that you can ship a tiny splash screen.

Requirements

Piece	Detail
Folders	Match the kit checklist (`projects/`, `notes/`, `labs/`)
`env_check.py`	Uses `sys.version` (or equivalent) and prints clearly
`hello.py`	At least two `print` lines
`my_intro.py`	At least three clear lines about you; optional `=` banner
Proof	`notes/day1.txt` plus a screenshot of `my_intro.py` running

Expected Behavior

- Terminal shows Python **3.x**
- Each script runs with no errors
- `my_intro.py` is about **you**, not a pasted sample name

Sample Run

```
Python version: 3.12.x ...
Setup OK — ready for Core Python!
```

```
Hello, World!
Welcome to Python Full Stack training.
```

```
====================================
  KSR — Day 1 Intro
====================================
Name: Priya Sharma
Goal: Become a full-stack Python developer
Why Python: Short syntax, strong for APIs later
====================================
```

Bonus Challenges

- Add a shebang (`#!/usr/bin/env python3`) and `chmod +x` on macOS/Linux

- Add `projects/welcome_batch.py` — 5-line KSR banner for your batch name
- Optional: `import keyword` then `print(len(keyword.kwlist))` — how many keywords your Python has

Done when

- [ ] Terminal shows Python **3.x**
- [ ] Folder layout matches the kit checklist
- [ ] `env_check.py` and `hello.py` run with no errors
- [ ] `my_intro.py` prints at least three clear lines about you
- [ ] `notes/day1.txt` exists with listing + version output
- [ ] Screenshot of running `my_intro.py` is ready to show

Interview Q&A

Q1. What is Python used for?
A high-level, readable language — scripts, automation, APIs, and web backends (this course's path).

Q2. Is Python compiled or interpreted?
Mainly **interpreted**: the interpreter runs your code. Behind the scenes it may use bytecode — for interviews, "interpreted / high-level" is enough.

Q3. What is a keyword? Give an invalid example.
A reserved word (`if`, `def`, `class`). Invalid: `class = 5` → `SyntaxError`.

Q4. How do you run a `.py` file?
`python3 filename.py` from the folder that contains the file (`python` on many Windows setups).

Q5. REPL vs script — when do you use each?
REPL for a quick experiment. Scripts for work you keep, share, and submit.

Q6. Why `snake_case`?
PEP 8 style for variables and functions — matches the standard library and team reviews.

Q7. What does `"=" * 24` do?
Repeats the character `=` twenty-four times — handy for banner borders.

Cheat Sheet

```
python3 --version     → prove Python 3 is installed
python3               → open the REPL (>>> )
exit()                → leave the REPL
python3 file.py       → run a script
print("hi")           → show text
print(2 + 3)          → show a calculated number
# comment             → ignored by Python
"=" * 40              → repeat a character (banner)
snake_case            → variable names (no spaces, no keywords)
f"Hi, {name}"         → embed a value in a string
import sys            → sys.version for an env check
```

Save before you run. `cd` into the folder that contains the file.

Completion Checklist

```
[ ] I can show Python 3.x from the terminal
[ ] I can run a line in the REPL and the same idea in a .py file
[ ] I can print text, a sum, and a = banner
[ ] I can choose snake_case names and avoid keywords
[ ] I can write a simple f-string greeting
[ ] I completed the welcome-card lab
[ ] I attempted the terminal-workspace mini-project
[ ] I can explain REPL vs script in an interview
```

What's Next?

You can now install Python, save a script, and print a welcome.

Next, Variables, Datatypes, Operators, and I/O teaches **labelled boxes** for values, math, `input()`, and clearer f-strings — so a script can remember a name and an amount, not only print fixed text.